

珠海天威微电子股份有限公司

PRB480 V0.0

1024位 1-Wire FRAM 防伪认证芯片

数 据 手 册



1Kb带SHA-1算法保护的 1-Wire FRAM 芯片

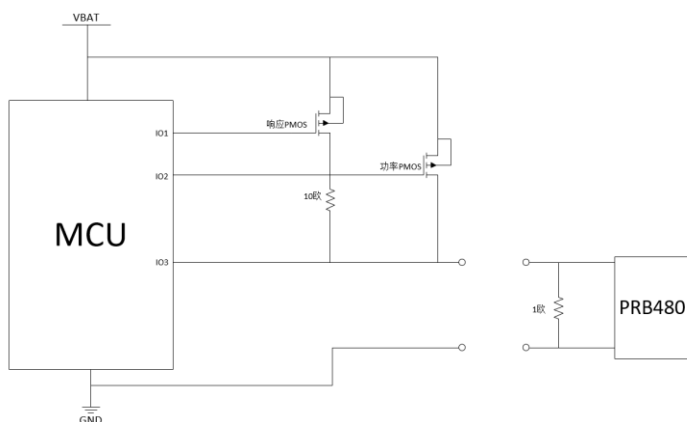
综述

PRB480芯片带有1024位FRAM存储器，通过ISO/IEC 10118-3安全散列算法（SHA-1）实现询问-应答的认证机制。1024位FRAM存储器分为4页，每页256位数据，通过64位暂存器实现写操作。每一页数据都可以设置写保护。每一颗PRB480芯片都有其独一的64位ROM注册码。PRB480芯片使用单总线通讯。通讯协议类似标准的1-Wire协议，但是为了在无电容的环境下使用，与标准的1-Wire协议有些差别。

应用场景

墨盒/碳粉打印盒识别
医疗传感器校准数据存储
消费类产品的售后管理

典型外围电路



管脚信息见文档末尾。

特点

- ◆ 1024位FRAM存储器分为4页，每页256位数据
- ◆ 片内集成512位输入的SHA-1算法引擎，用来计算160位信息摘要认证码(MACs)以及生成密钥
- ◆ 包含64位只写密钥，以及1页256位数据可以设置读、写保护
- ◆ 写入芯片时，必须持有密钥，正确计算并传输160位MAC来实现认证
- ◆ 用户可以设置写保护第0页，第3页或者全部4页
- ◆ 单总线通讯速率在125kbps左右
- ◆ 读写的工作电压在2.4V 到3.65V内，工作温度在-40°C 到 +85°C
- ◆ 封装信息待补充

其他信息

1Kb带SHA-1算法保护的 1-Wire FRAM 芯片

电气特性参数表

(温度在 -40°C 到 +85°C.)

参数	符号	条件	最小	典型	最大	单位
IO 管脚:通用数据						
单总线上拉电压	V _{PUP}		2.4		3.65	V
输入负载电流 Input Load Current	I _L	IO pin at V _{PUP}		5		μA
高到低 转换阈值电压 High-to-Low Switching Threshold	V _{TL}		0.46		V _{PUP} - 1.8	V
输入低电压 Input Low Voltage	V _{IL}				0.5	V
低到高 转换阈值电压 Low-to-High Switching Threshold	V _{TH}		1.0		V _{PUP} - 1.1	V
Switching Hysteresis	V _{HY}			500		mV
输出电流	I _{OL}			300		mA
时隙	t _{SLOT}	速度125Kbits/s		30		μs
IO 管脚:单总线复位						
复位低电平时间	t _{RSTL}	速度125Kbits/s		300		μs
上电复位后等待系统稳定的时间	t _{STD}	速度125Kbits/s		200		μs



1Kb带SHA-1算法保护的 1-Wire FRAM 芯片

电气特性参数表

(温度在 -40°C 到 +85°C.)

参数	符号	条件	最小	典型	最大	单位
IO 管脚: 单总线写						
写逻辑1, 低脉冲低电平持续时间	t_{LOW}	速度125Kbits/s		1		μs
写逻辑0, 两个低脉冲间隔时间	t_{GAP}	速度125Kbits/s		1		μs
IO 管脚: 单总线读						
拉低总线后到开始采样前的时间	t_1	速度125Kbits/s		2		μs
采样后到总线上拉的时间	t_2			1		μs
读0下拉时间	t_{READ0}	速度125Kbits/s		5		μs
FRAM						
写电流	I_{PROG}			5	6	mA
写时间	t_{PROG}		100			ns
写周期	N_{CY}	+25°C	400k			次
		+85°C	100k			
数据有效时间	t_{DR}	+85°C	1			年
SHA-1算法						
算法时电流	I_{LCSHA}	$V_{PUP} = 3.3V$			TBD	mA
算法计算时间	t_{CSHA}	-5°C 到 +85°C			2.0	ms
		-40°C 到 +85°C			2.15	

1Kb带SHA-1算法保护的 1-Wire FRAM 芯片

管脚定义

管脚名称	功能
PAD1	参考地或单总线，支持正反插
PAD2	参考地或单总线，支持正反插

具体描述

综述

一颗PRB480芯片中有1024位FRAM存储器，其中包含4页256位数据，一个64位密钥，一页配置数据，一个512位输入的SHA-1算法引擎和一个64位ROM注册码。数据通过单总线传输，通讯仅需一根数据线和一根地线。PRB480有一片称作暂存器的额外存储区，当写入用户区数据，配置页数据或者加载一个新的密钥时，暂存器区域可供数据缓冲。数据首先写入到暂存器中，且可以从暂存器中读出来。当数据验证通过后，使用Copy Scratchpad 命令并提供正确的160位MAC，数据最终就可以写入到目标地址。

MAC的计算需要密钥，部分芯片内部存储的数据，以及64位ROM注册码。只有载入一个新密钥时不需要提供MAC。

芯片的64位ROM注册码保证了每一颗芯片的唯一性。PRB480的应用包括附件/PC 板识别，医疗传感器校准数据存储，模拟传感器校准，墨盒/碳粉打印盒识别，以及消费类产品的售后管理。

图 1 中的结构框图说明了 PRB480主控单元与存储器部分的关系。图2说明了通讯协议的层次结构。主机必须首先发送如下五条 ROM 操作命令中的一条：1) Read ROM, 2) Match ROM, 3) Search ROM, 4) Skip ROM, 5) Resume。与 ROM 操作命令有关的协议说明见图 10。成功地执行了 ROM 操作命令后，就可以进行存储器或者SHA-1操作，这些操作功能见图8。所有数据读写时，都是低位在前。



1Kb带SHA-1算法保护的 1-Wire FRAM 芯片

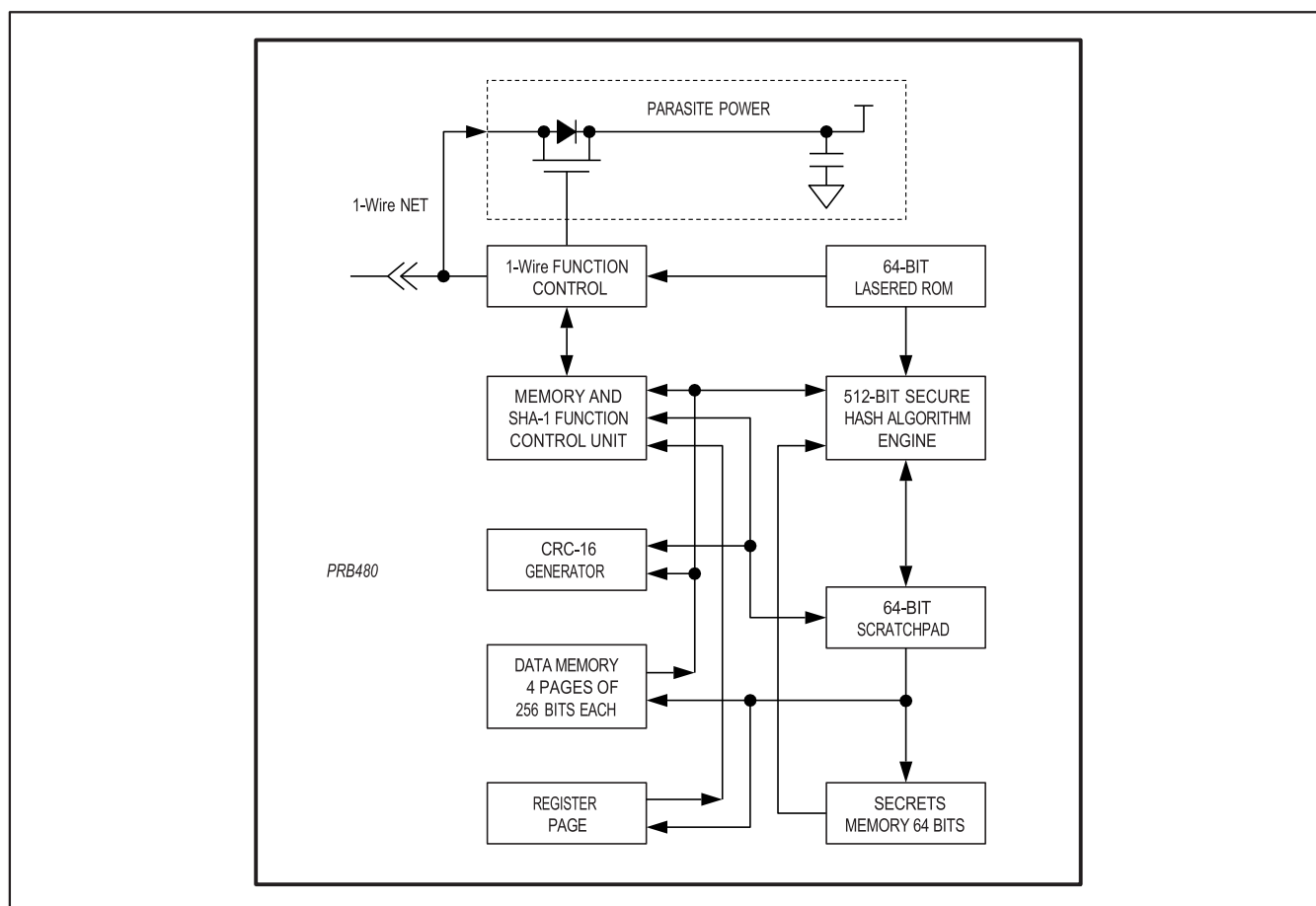


图 1. 结构框图

64位ROM注册码

每个PRB480 都有唯一一个 64 位ROM码，其中前 8 位是 1-Wire家族码，中间 48 位是唯一的序列号，最后 8位是前 56 位的CRC（循环冗余校验）码。详见图 3 所示。1-Wire CRC 校验码通过一个包括移位寄存器和异或门的多项式发生器产生，如图 4 所示。该多项式为： $X^8 + X^5 + X^4 + 1$ 。

移位寄存器初始化时被清 0，然后从家族码的最低有效位开始，每次移入一位。当家族码的最后一位被移入后，再移入序列号。当序列号的最后一位也被移入时，移位寄存器的值即为 CRC 码的值。继续移入 8 位 CRC 码后，移位寄存器所有位归 0。

存储器访问

PRB480有四片存储器区域：数据区，密钥区，配置区(包含特殊功能的寄存器配置和一些用户字节)，暂存器区。数据区有4页，每页数据有32字节。密钥区和暂存器区都是8字节长度。暂存器用来作为写入数据区，配置区以及密钥区的缓冲区域。

数据存储器和寄存器位于一个线性地址空间，如图 5 所示。写入数据区和配置区需要持有密钥。密钥通过暂存器的数据写入，密钥不能直接读取；只有SHA-1算法引擎在计算HAMC时会访问密钥。

1Kb带SHA-1算法保护的 1-Wire FRAM 芯片

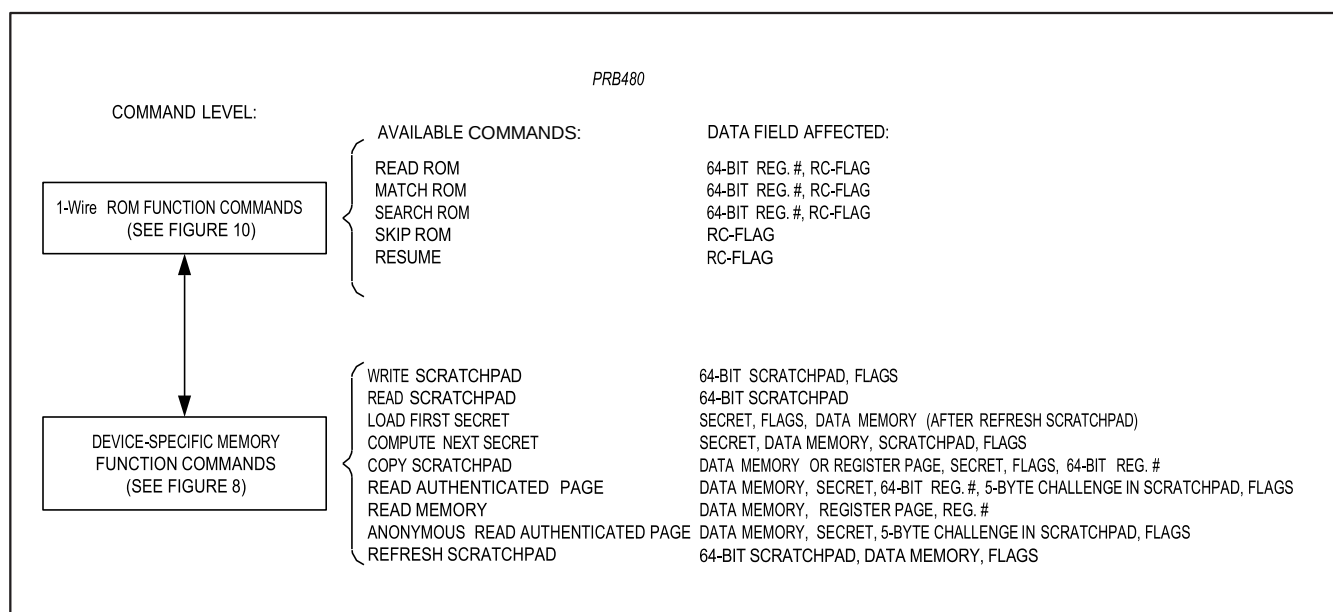


图 2. 命令层次图

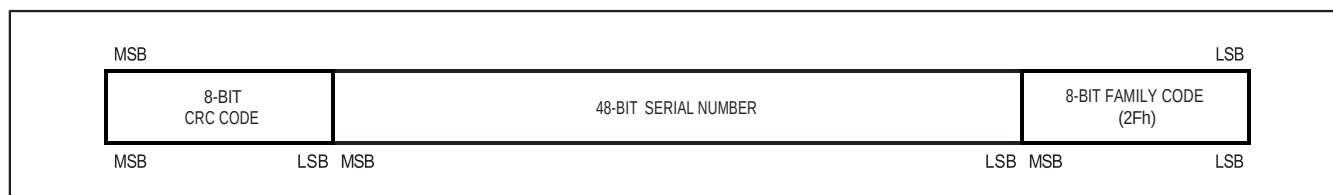


图 3. 64-位ROM注册码

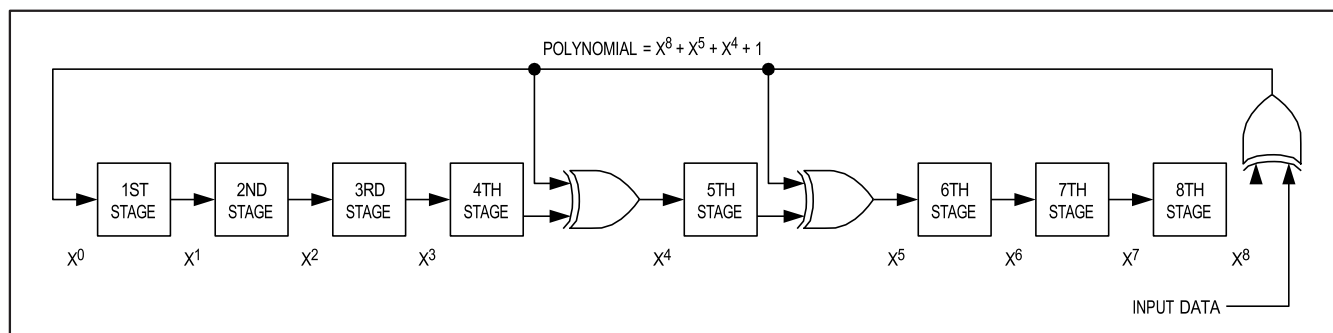


图 4. 1-Wire CRC 校验值生成

地址范围0088-009Fh为寄存器配置，包含特殊功能的寄存器数据、用户字节和工厂字节。一旦被写入AAh或者55h后，这些数据大部分都会被写保护，并且从此再也不能被改写。

图6表明了每个配置对数据区，密钥区和其他配置区的影响。配置区的特殊功能包括写保护密钥，写保护地址8Ch-8Fh，同时写保护四页数据区，单独写保护页0，单独写保护页3，以及读保护页3。

1Kb带SHA-1算法保护的 1-Wire FRAM 芯片

读保护页3可以将整个页3的数据变成密钥的一部分，从而达到更进一步的安全认证。

工厂字节只会是55h或者AAh。当工厂字节为55h时，表明008Eh和008Fh的地址数据为用户字节，可供用户读写；当工厂字节是AAh时，表明008Eh和008Fh的地址数据已经被写为16位的制造商ID且已经被写保护。

地址0090h–0097h的数据为64位ROM注册码，主机可以通过ROM命令正确地读取全部64位ROM注册码。

写入FRAM分为两步，首先用Write Scratchpad命令写入暂存器，再用Copy Scratchpad命令将暂存器数据复制到FRAM的目标地址中。在复制暂存器数据之前，可以用Read Scratchpad命令验证写入的数据是否正确。

如果目标地址被写保护了，Copy Scratchpad命令不会生效。

ADDRESS RANGE	TYPE	DESCRIPTION
00h–1Fh	R/W	用户数据第0页
20h–3Fh	R/W	用户数据第1页
40h–5Fh	R/W	用户数据第2页
60h–7Fh	(R)/(W)	用户数据第3页
80h–87h	W	密钥数据，不可读
88h	R/(W)	写保护密钥以及8Ch到8Fh的数据
89h	R/(W)	写保护用户数据第0到第3页
8Ah	R/(W)	保留数据，写成55h或AAh后写保护自己
8Bh	Read	保留数据
8Ch	R/(W)	保留数据，不要写成55h或AAh(Note2)
8Dh	R/(W)	写保护用户数据第0页
8Eh–8Fh	R/(W)	保留数据
90h–97h	Read	64位ROM注册码，只读
98h	R/(W)	写保护用户数据第3页
99h	R/(W)	读保护用户数据第3页
9Ah–9Fh	Read	保留数据。

图5. 存储器结构图



1Kb带SHA-1算法保护的 1-Wire FRAM 芯片

CONTROLLING REGISTER ADDRESS (Note 1)	AFFECTED MEMORY AREA						
	PAGE 0 (00h–1Fh)	PAGE 1 (20h–3Fh)	PAGE 2 (40h–5Fh)	PAGE 3 (60h–7Fh)	SECRET (80h–87h)	8Ch, 8Dh	8Eh, 8Fh
88h	—	—	—	—	Write protect	Write protect	Write protect
89h	Write protect	Write protect	Write protect	Write protect	—	—	—
8Bh	—	—	—	—	—	—	—
8Ch	—	—	—	—	—	—	—
8Dh	Write protect	—	—	—	—	—	—
98h	—	—	—	Write protect	—	—	—
99h	—	—	—	Read protect	—	—	—

Note 1: 如果没有特别说明，一片存储器的数据在它的控制寄存器写为55h或AAh后会被保护，控制寄存器本身在写为55h或AAh后也会被保护。

Note 2: 8Ch数据不要写成55h或AAh，否则可能出现未验证完毕的情况，55h和AAh以外的其他值可以随意写入。

图6. 存储器保护对照表

	BIT #	7	6	5	4	3	2	1	0
TARGET ADDRESS (TA1)		T7	T6	T5	T4	T3	T2 (0)	T1 (0)	T0 (0)
TARGET ADDRESS (TA2)		T15	T14	T13	T12	T11	T10	T9	T8
ENDING ADDRESS WITH DATA STATUS (E/S)		AA	1	PF	1	1	E2 (1)	E1 (1)	E0 (1)

图 7. 地址寄存器

地址寄存器和传输状态

PRB480 使用 3 个地址寄存器:TA1, TA2, 以及 E/S (见图 7)。寄存器 TA1 和 TA2 必须加载进行数据写入或读出的目标存储器地址。寄存器 E/S 是一个只读的传输状态寄存器，用来校验写操作命令的输入数据完整性。

由于每次暂存器只接收8字节数据，因此TA1的低3位强制为0，E/S寄存器的低3位固定为1。

E/S 寄存器的第5 位，称作 PF，如果暂存器数据因掉电或主机发送的数据不为整8字节长度而无效，该位被置为逻辑1。PF为0表明写暂存器操作正确无误。第 3, 4, 6 位没有功能；总是读出1。 主机可以通过PF校验写入数据的完整性。E/S 寄存器的最高位，称为 AA或授权许可，作为指示暂存器数据已被复制到目标存储器地址的标志位，向暂存器中写入数据将清除此位。



1Kb带SHA-1算法保护的 1-Wire FRAM 芯片

带验证的写操作

向 PRB480 写入数据时，必须把暂存器用作中间存储器，首先主机发送 **Write Scratchpad** 命令并指定目标存储器地址，随后发送要写入暂存器的数据。注意 **Copy Scratchpad** 命令指定的地址必须是行首的地址，也就是说，目标地址的低3位T[2:0]必须等于000b。如果发送的T[2:0]不为000b，芯片也会将这些位置为0。主机每次都应该传输8个完整的字节作为写入的数据。在8字节数据传输后，主机可以接受到一个CRC-16的反码，它由**Write Scratchpad**命令、主机发送的地址以及主机发送的8字节数据计算生成。主机同样计算出一个CRC值，与接收到的进行对比，从而判断这次通讯是否成功。写入暂存器后，主机应该立刻执行**Read Scratchpad**命令来确认写入数据是否正确。**Read Scratchpad**命令期间，主机重复发送目标地址TA1和TA2，以及E/S寄存器。如果暂存器数据掉电丢失或主机发送的数据不完整，PF(E/S寄存器的bit 5)位被置为逻辑 1。授权标志AA(E/S寄存器的bit 7)一般会被**Write Scratchpad**命令清0；因此如果授权标志被指为1，表明芯片不接受当前进行的**Write Scratchpad**命令。不管是哪一种出错的情况，主机都应该重新写暂存器。

接收到E/S寄存器后，主机会继续接受到暂存器的数据，接着接收到一个CRC的反码，该CRC由**Read Scratchpad**命令，目标地址，E/S寄存器计算产生。和**Write Scratchpad**命令一样，主机通过对比CRC来判断通讯是否成功。主机确认数据后，可以发送**Copy Scratchpad**命令将暂存器数据复制到FRAM中。后续命令介绍有更多详细信息。

在某些极端不稳定的使用场景下，电子通讯过程的有效性可能无法保障。在**Copy Scratchpad**命令的过程中，由于电量的不足，可能使得存储器的某个bit的电压域趋于0和1之间。这种情况下，这个bit的逻辑值不确定，在验证SHA-1算法的过程中，主机读取到的值和芯片内部SHA-1引擎读取到的值可能相反，从而MAC计算值不相同，算法验证不通过。为了避免上述情况的发生，芯片设计了**Refresh Scratchpad**命令，与**Load First Secret**命令相结合，可以绕过算法验证来刷新存储器的数据。

在脆弱敏感的环境下，在**Copy Scratchpad**命令写入数据后应该进行一次刷新的过程。使用**Refresh Scratchpad**命令，指定相同的地址，再接着**Load First Secret**命令，指定同样的地址，数据从FRAM存储器中读出，写入到暂存器，又立即回写到原来的地址。**Refresh Scratchpad**命令会将EN_LFS标志置1，从而使**Load First Secret**命令可以指定00h-7F的地址，这样的写入不需要通过MAC的计算。在**Refresh Scratchpad**之后的任意改变暂存器数据的命令会将EN_LFS标志置0。这样会保证数据只是被刷新而不会被别的数据覆盖。**Refresh Scratchpad**命令指定的地址在80h及以上时，该命令与**Write Scratchpad**命令效果一致，且EN_LFS标志不会被置1，因此不能刷新密钥数据(80h)和配置数据(88h及以上)。这样可以防止密钥在跟着**Refresh Scratchpad**命令后的**Read Scratchpad**命令中泄露。

1Kb带SHA-1算法保护的 1-Wire FRAM 芯片

存储器命令和SHA-1功能命令

作为加密芯片，任何读取密钥或者读保护数据的行为都只会读取到FF字节而不是真实数据。存储器命令和SHA-1功能命令流程图(图8)描述了访问存储器以及使用SHA-1算法引擎的流程。主机和PRB480之间的通讯速率默认为125kbps。

Write Scratchpad [0Fh]

Write Scratchpad(写暂存器)命令适用于数据存储区、密钥存储区和寄存器页中的可写地址。该命令会将EN_LFS标志置0。

发出 Write Scratchpad 命令后，主机必须首先发送 2 个字节的地址，接着发送要写入暂存器的数据。写入数据从暂存器第一个字节开始，后面每收到一个数据字节加 1。E[2:0](如图7)始终为111b，不管主机传输多少字节数据。因此为了保证暂存器中的数据能够被正确复制到存储器阵列中，主机应该总是发送8字节整的数据。

如果主机发送了少于8字节的数据，后续又没有读出暂存器的数据来验证，就将数据载入作为密钥，那么部分密钥就会未知。每次必须发送整8字节数据。如果最后一字节的数据不完整，那么它的内容被忽略，且PF标志置1。

当执行 Write Scratchpad 命令时，PRB480 内部的CRC发生器(图 14)随着主机的发送过程，计算整个数据流的CRC校验码，始于命令代码，终止于最后一个数据字节。该CRC校验码由CRC16多项式生成，计算时首先清除CRC发生器，然后顺序移入 Write Scratchpad 命令代码(0Fh)，目标地址(TA1和TA2)，和所有数据字节。要注意的是，CRC16 计算时使用的是由主机实际发送的 TA1、TA2 和数据字节，尽管芯片会将T[2:0]置为000b。

主机可在任意时间终止 Write Scratchpad 命令。只要写入数据达到暂存器上限(E[2:0]=111b)，主机就可发送16个读时隙并收到 PRB480产生的CRC校验码。读完CRC后继续读取，会接收到FFh。

如果 Write Scratchpad命令试图对写保护区域进行写入，暂存器将加载存储器原有的数据，而不是主机发送的数据。

Read Scratchpad [AAh]

Read Scratchpad(读暂存器)命令可以用来校验目标地址和暂存器数据的完整性。主机发送命令代码后开始读取数据。开头的两个字节是目标地址，下一个字节是E/S寄存器，接着是暂存器数据，这些数据可能与主机发送的原始数据有所不同。当目标地址位于寄存器页或位于写保护时，这一点尤其重要。详细信息见Write Scratchpad命令说明。主机应先读完暂存器中所有数据，然后就可以收到反码的CRC，该CRC码根据PRB480发送的数据产生。如果主机在收到CRC码后继续读取数据，得到的所有数据均为逻辑1。

Load First Secret [0Fh]

Load First Secret(加载初始密钥)命令在EN_LFS标志不同的情况下，执行不同的操作。EN_LFS=0时，如果密钥没有写保护，芯片会将当前暂存器的数据写入密钥；EN_LFS=1时，该命令会跳过SHA-1算法，重写数据存储区(00h-7Fh地址)。EN_LFS标志默认为0，除非执行了Refresh Scratchpad命令。

如果 EN_LFS = 0

在执行Load First Secret命令之前，主机必须先执行Write Scratchpad命令，设置目标地址为密钥区(80h)，将新的密钥写入到暂存器中。执行Load First Secret命令，主机需要发送3字节的认证数据(TA1,TA2,E/S,依次顺序)，这些数据可以在Write Scratchpad命令后的Read Scratchpad命令中获取，这三字节数据必须完全匹配。如果密钥没有写保护，AA标志置1，然后8字节数据写入密钥区。

如果 EN_LFS = 1

同样的，这种情况下也需要在执行Load First Secret命令之前，用Refresh Scratchpad命令将数据写入到暂存器(地址范围00h-7Fh)中，此时EN_LFS被置1。执行Load First Secret命令，主机需要发送3字节的认证数据(TA1,TA2,E/S,依次顺序)，这些数据可以在Refresh Scratchpad命令后的Read Scratchpad命令中获取，Read Scratchpad命令不会将EN_LFS置0，这三字节数据必须完全匹配。如果选中的区域数据没有写保护，AA标志置1，然后8字节数据写入选中区域数据。

不管是哪一种情况下，复制暂存器数据时的单总线电压不能低于2.4V。主机可以在复制完成后读取总线上的应答数据，读取到AAh表明复制成功，读取到FFh表明复制失败。

1Kb带SHA-1算法保护的1-Wire FRAM芯片

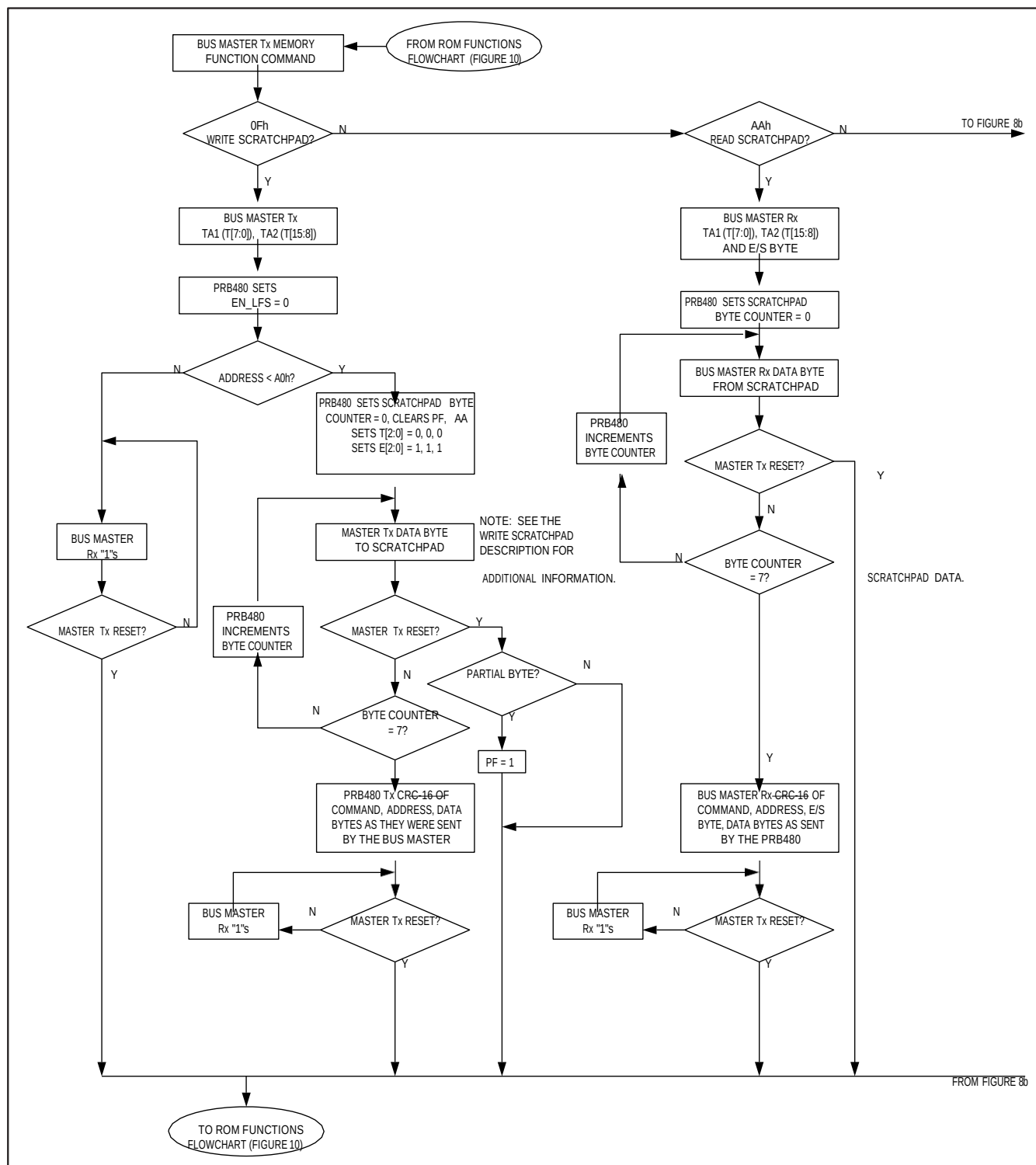


图 8a. 存储器 and SHA-1 功能流程图

1Kb带SHA-1算法保护的1-Wire FRAM芯片

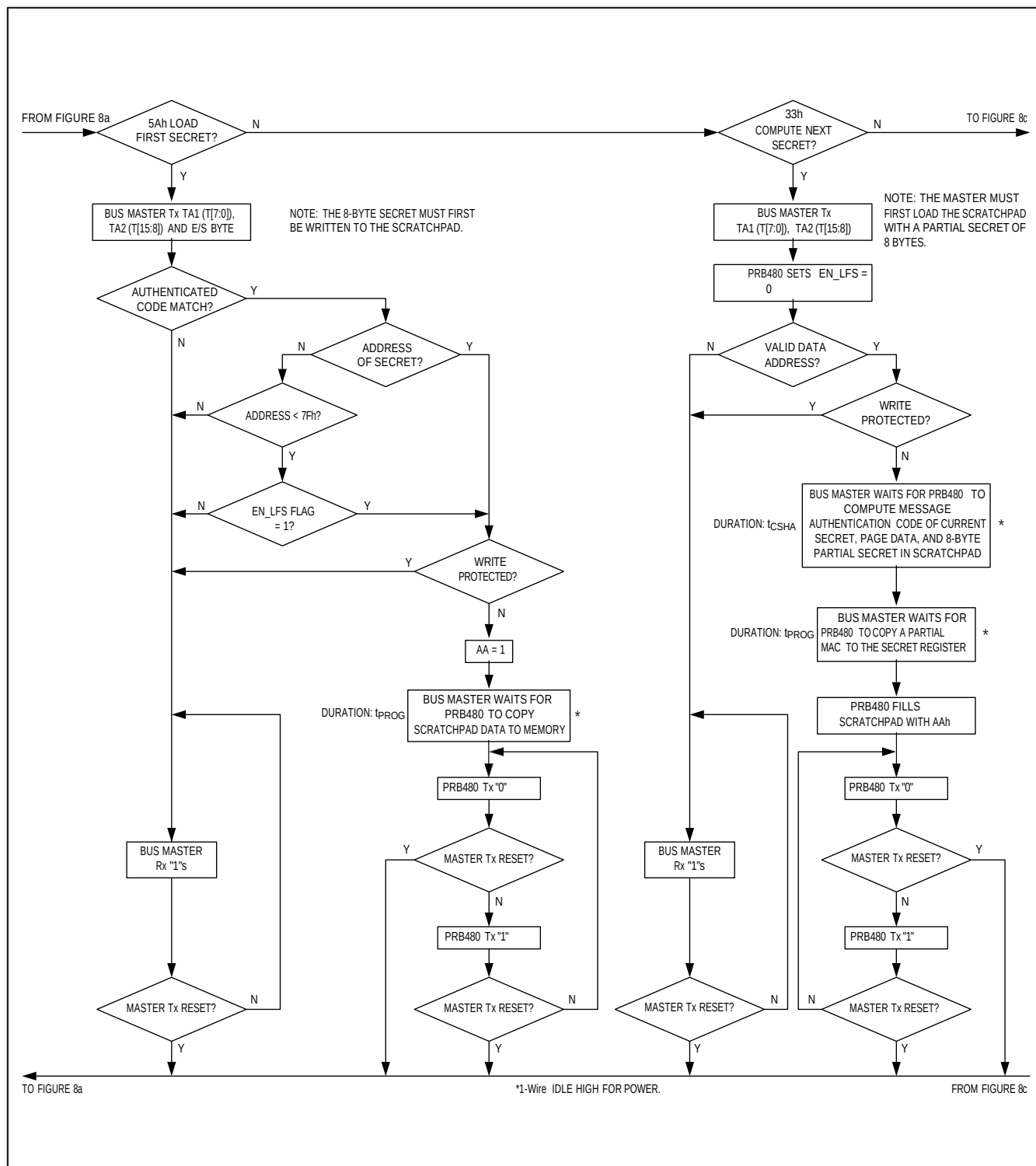


图 8b. 存储器 and SHA-1 功能流程图(续)

1Kb带SHA-1算法保护的1-Wire FRAM芯片

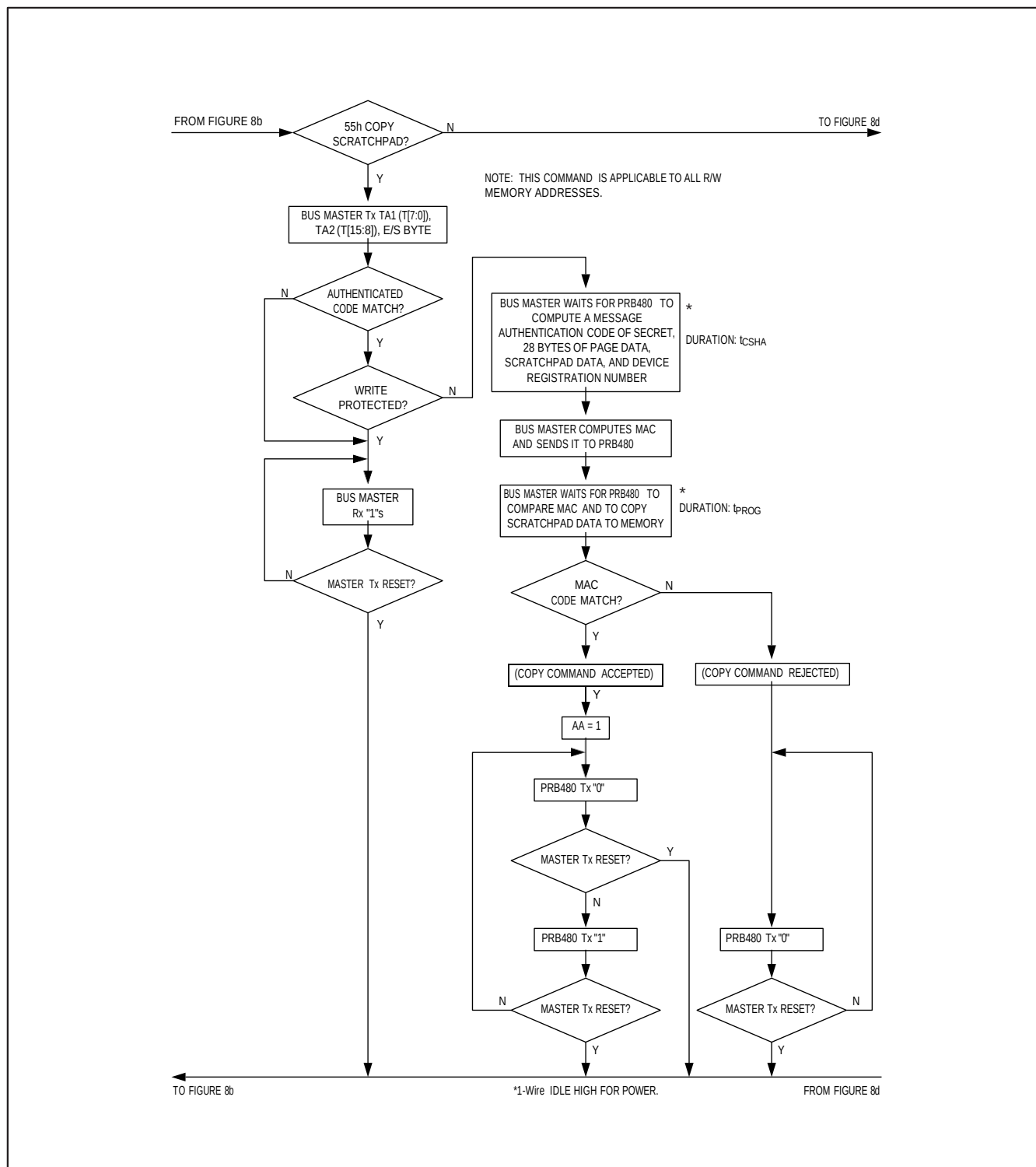


图 8c. 存储器 and SHA-1 功能流程图 (续)

1Kb带SHA-1算法保护的1-Wire FRAM芯片

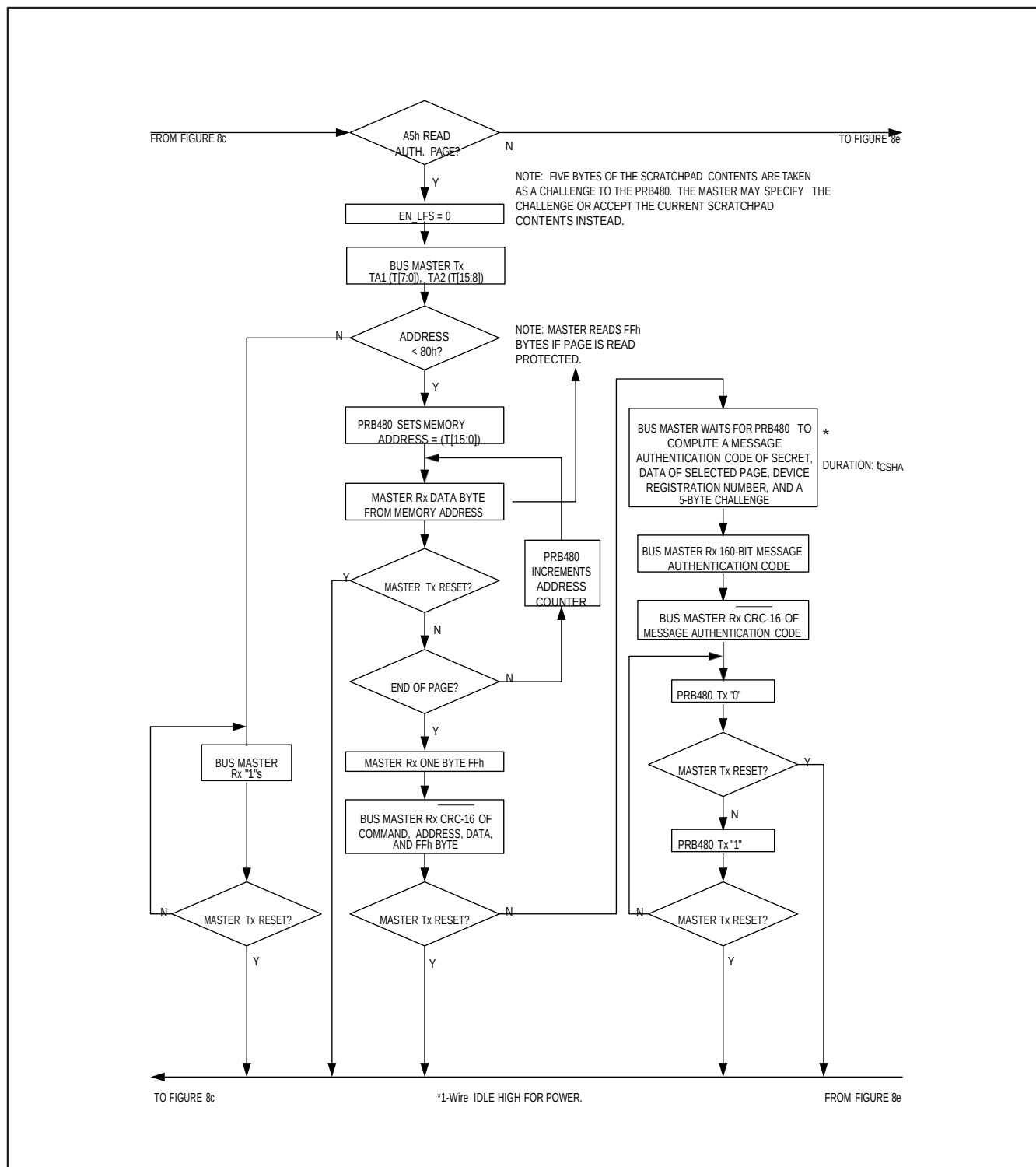


图 8d. 存储器 and SHA-1 功能流程图 (续)

1Kb带SHA-1算法保护的1-Wire FRAM芯片

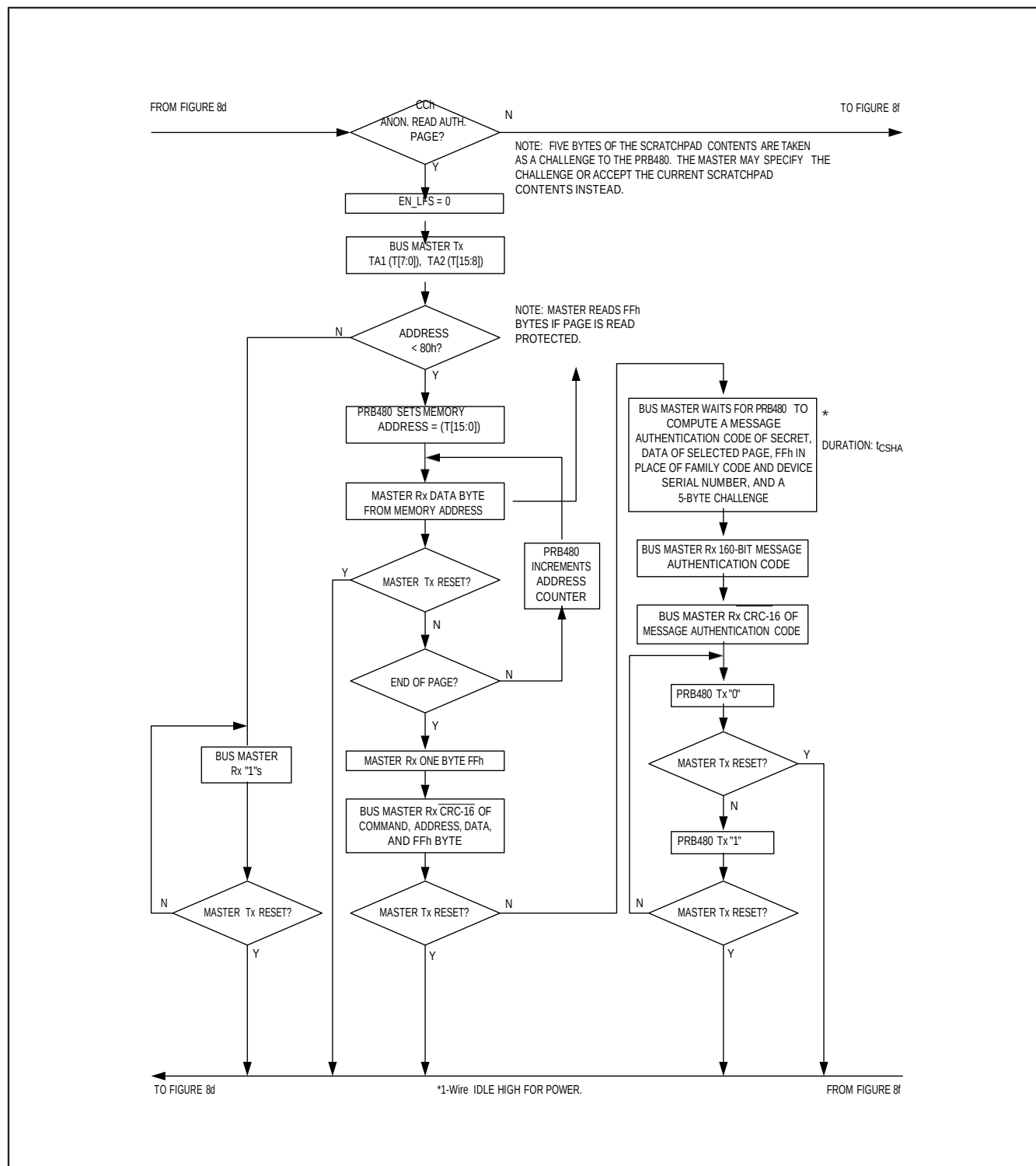


图 8e. 存储器 and SHA-1 功能流程图 (续)

1Kb带SHA-1算法保护的1-Wire FRAM芯片

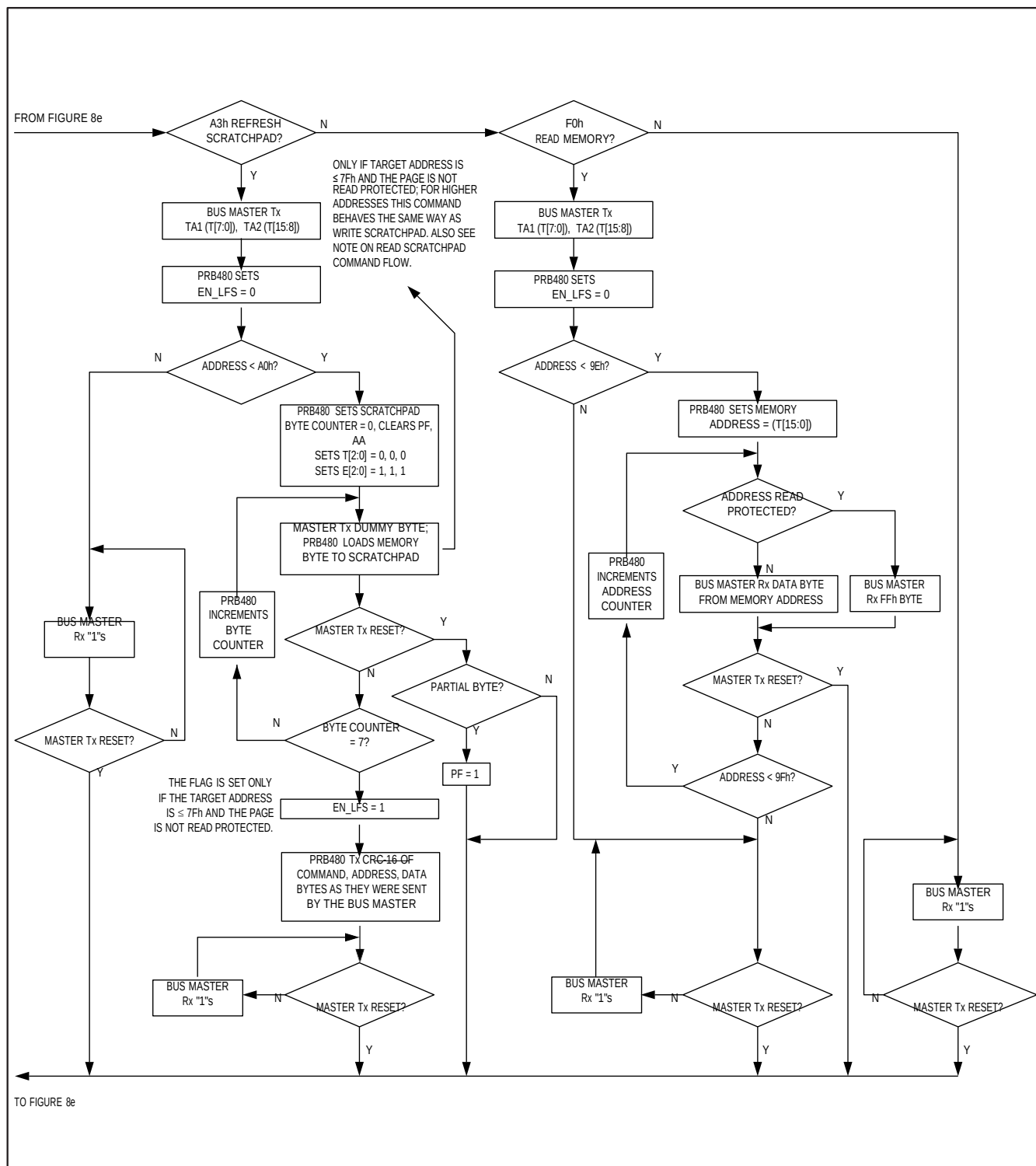


图 8f. 存储器和SHA-1功能流程图 (续)

1Kb带SHA-1算法保护的1-Wire FRAM芯片

FRAM认证命令[DDC0h,DEC0h]

FRAM认证命令在上电后使用，上电后首先发送FFFF8C73h进入认证模式。该命令用于在拔插芯片后，快速地写读同一个地址来验证FRAM的快写能力，起到防伪的作用。读认证命令为DDC0h，写认证命令为DEC0h。主机首先发送一次写命令，指定地址C000h和写入的数据，写入数据为8字节，接着在10us内立刻掉电，因为FRAM写入数据的速度很快，如果是别的存储器例如EEPROM或者FLASH存储器，短时间内数据不能完全写入；重新上电复位后，再次读取目标地址C000h的数据，如果读取到的数据和上一次写入的完全一致，那么认证通过，当前使用的芯片是FRAM芯片。用于验证的地址C000h的数据属于无效的数据，既不在数据区，也不在配置区，仅用于验证快速写读的正确性，数据写入也不需要SHA-1算法。验证通过后，芯片要重新掉电以退出认证模式，从而执行其他正常的命令。

Compute Next Secret [33h]

在某些应用场景下可能需要更高安全性的密钥，单一的，直接写入的密钥无法满足需求，因此PRB480提供了Compute Next Secret(生成下一密钥)命令，这一命令根据当前的密钥，选中的存储数据，以及暂存器里的数据生成一个新的密钥。

启动这一命令需要密钥数据没有写保护，表1展示了各种数据是如何参与到SHA-1算法的输入中的，以及算法的结果是如何构成新密钥的。SHA-1算法在后续会介绍。这一命令可以多次使用来不断提高安全等级。

表 1. Compute Next Secret 命令的SHA-1输入数据

M0[31:24] = (SS + 0)	M0[23:16] = (SS + 1)	M0[15:8] = (SS + 2)	M0[7:0] = (SS + 3)
M1[31:24] = (PP + 0)	M1[23:16] = (PP + 1)	M1[15:8] = (PP + 2)	M1[7:0] = (PP + 3)
M2[31:24] = (PP + 4)	M2[23:16] = (PP + 5)	M2[15:8] = (PP + 6)	M2[7:0] = (PP + 7)
M3[31:24] = (PP + 8)	M3[23:16] = (PP + 9)	M3[15:8] = (PP + 10)	M3[7:0] = (PP + 11)
M4[31:24] = (PP + 12)	M4[23:16] = (PP + 13)	M4[15:8] = (PP + 14)	M4[7:0] = (PP + 15)
M5[31:24] = (PP + 16)	M5[23:16] = (PP + 17)	M5[15:8] = (PP + 18)	M5[7:0] = (PP + 19)
M6[31:24] = (PP + 20)	M6[23:16] = (PP + 21)	M6[15:8] = (PP + 22)	M6[7:0] = (PP + 23)
M7[31:24] = (PP + 24)	M7[23:16] = (PP + 25)	M7[15:8] = (PP + 26)	M7[7:0] = (PP + 27)
M8[31:24] = (PP + 28)	M8[23:16] = (PP + 29)	M8[15:8] = (PP + 30)	M8[7:0] = (PP + 31)
M9[31:24] = FFh	M9[23:16] = FFh	M9[15:8] = FFh	M9[7:0] = FFh
M10[31:24] = MPX	M10[23:16] = (SP + 1)	M10[15:8] = (SP + 2)	M10[7:0] = (SP + 3)
M11[31:24] = (SP + 4)	M11[23:16] = (SP + 5)	M11[15:8] = (SP + 6)	M11[7:0] = (SP + 7)
M12[31:24] = (SS + 4)	M12[23:16] = (SS + 5)	M12[15:8] = (SS + 6)	M12[7:0] = (SS + 7)
M13[31:24] = FFh	M13[23:16] = FFh	M13[15:8] = FFh	M13[7:0] = 80h
M14[31:24] = 00h	M14[23:16] = 00h	M14[15:8] = 00h	M14[7:0] = 00h
M15[31:24] = 00h	M15[23:16] = 00h	M15[15:8] = 01h	M15[7:0] = B8h

1Kb带SHA-1算法保护的1-Wire FRAM芯片

Compute Next Secret 的结果

$(SS + 0) := E[7:0]$	$(SS + 1) := E[15:8]$	$(SS + 2) := E[23:16]$	$(SS + 3) := E[31:24]$
$(SS + 4) := D[7:0]$	$(SS + 5) := D[15:8]$	$(SS + 6) := D[23:16]$	$(SS + 7) := D[31:24]$

注释:

Mt	SHA-1的输入; $0 \leq t \leq 15$; 32-位数
$(SS + N)$	密钥的第N个字节; 密钥从地址80h开始(见图5)
$(PP + N)$	数据页的第N个字节; 数据页从00h, 20h, 40h以及60h开始(见图5)
$(SP + N)$	暂存器的第N个字节
MPX	$MPX[7] = 0$; $MPX[6] = 0$; $MPX[5:0] = (SP + 0)[5:0]$
D, E	32位数, 来自160位SHA-1结果

启动Compute Next Secret命令后, 主机继续发送2字节地址来指定一页数据作为SHA-1的输入。接收到地址后, 芯片将EN_LFS标志清0。由于指定的是一整页数据, 地址TA1的低五位无效。指定的地址是有效的话(在00h到7Fh范围内), 且密钥没有被锁定, SHA-1算法开始计算。

主机等待 tCSHA 的算法计算时间, 算法结束后, 接着立刻开始将新的密钥载入到密钥区域中, 载入过程需要等待 tPROG 时间。在tCSHA 和tPROG 时间内, 单总线上的电压不能低于2.4V。

密钥载入成功后, 暂存器的值被全部置为AAh。如果由于指定地址无效或者密钥写保护出现错误, 暂存器的值就不会改变。主机可以在等待时间结束后读取1字节数据, 如果读取到AAh, 则表示密钥载入成功; 如果读取到FFh, 则表明密钥载入失败, 可能原因是指定地址无效或者密钥写保护。

由于暂存器的数据也会用作算法的计算, 因此在使用Compute Next Secret command命令之前, 主机必须使用Write Scratchpad命令来向暂存器中写入一个确定的8字节数据。

Copy Scratchpad [55h]

Copy Scratchpad(复制暂存器) 命令需要持有芯片的密钥, 并且能够执行SHA-1算法来生成160位MAC, 之后向芯片发送正确的160位MAC值就能启动复制操作。

表2展示了MAC值传输的顺序和SHA-1结果的关系。表3A和表3B展示了不同情况下, SHA-1的输入有所不同。SHA-1算法详细的数学计算公式在文档后面会有介绍。启动Copy Scratchpad命令后, 主机必须提供3字节认证数据, 这些认证数据可以通过Read Scratchpad命令获取, 它们依次为TA1, TA2和E/S字节。如果认证字节正确且目标地址没有被写保护, 那么芯片将会开始160MAC的计算, 计算输入包括当前的密钥, 目标地址页的28字节数据, ROM注册码的前7个字节以及暂存器的全部8字节数据(见表3A)。计算持续时间为tCSHA, 持续时间内单总线的电压不能低于2.4V。与此同时, 主机也进行一样的计算, 在tCSHA持续时间结束后, 将计算的MAC值发送给芯片做认证。

表 2. MAC传输顺序

E[31:24]	E[23:16]	E[15:8]	E[7:0]
D[31:24]	D[23:16]	D[15:8]	D[7:0]
C[31:24]	C[23:16]	C[15:8]	C[7:0]
B[31:24]	B[23:16]	B[15:8]	B[7:0]
A[31:24]	A[23:16]	A[15:8]	A[7:0]

注: 传输从E的最高位开始。

估计是从低位开始

1Kb带SHA-1算法保护的1-Wire FRAM芯片

芯片接收到主机发送的MAC值，与自己计算出来的MAC值相比较，两者完全一致的话，则认证通过，芯片开始将暂存器的数据复制到FRAM内，写入的过程持续tPROG，在这段时间内，单总线的电压不能低于2.4V。

在tPROG时间结束后，主机可以读取1字节数据，如果读取到AAh，则表明复制成功；如果读取到00h，则表明复制失败，原因是芯片计算的MAC值和接收到主机发送的MAC值不能完全匹配；如果读取到FFh，则表明复制失败，原因是目标地址被写保护或者发送的3字节认证数据错误。

当向配置寄存器区域写入数据时，需要特别注意，因为配置寄存器区域的数据一旦锁定就不能再次被改写。为了防止某些数据被意外锁定，最好写入之前先读取该部分的数据，并且确定无误地写入暂存器。当向配置寄存器页复制数据时，密钥会出现两次，M0/M12以及M1/M2，如表3B所示。

表 3A. Copy Scratchpad 命令用于数据页时的SHA-1输入

M0[31:24] = (SS + 0)	M0[23:16] = (SS + 1)	M0[15:8] = (SS + 2)	M0[7:0] = (SS + 3)
M1[31:24] = (PP + 0)	M1[23:16] = (PP + 1)	M1[15:8] = (PP + 2)	M1[7:0] = (PP + 3)
M2[31:24] = (PP + 4)	M2[23:16] = (PP + 5)	M2[15:8] = (PP + 6)	M2[7:0] = (PP + 7)
M3[31:24] = (PP + 8)	M3[23:16] = (PP + 9)	M3[15:8] = (PP + 10)	M3[7:0] = (PP + 11)
M4[31:24] = (PP + 12)	M4[23:16] = (PP + 13)	M4[15:8] = (PP + 14)	M4[7:0] = (PP + 15)
M5[31:24] = (PP + 16)	M5[23:16] = (PP + 17)	M5[15:8] = (PP + 18)	M5[7:0] = (PP + 19)
M6[31:24] = (PP + 20)	M6[23:16] = (PP + 21)	M6[15:8] = (PP + 22)	M6[7:0] = (PP + 23)
M7[31:24] = (PP + 24)	M7[23:16] = (PP + 25)	M7[15:8] = (PP + 26)	M7[7:0] = (PP + 27)
M8[31:24] = (SP + 0)	M8[23:16] = (SP + 1)	M8[15:8] = (SP + 2)	M8[7:0] = (SP + 3)
M9[31:24] = (SP + 4)	M9[23:16] = (SP + 5)	M9[15:8] = (SP + 6)	M9[7:0] = (SP + 7)
M10[31:24] = MP	M10[23:16] = (RN + 0)	M10[15:8] = (RN + 1)	M10[7:0] = (RN + 2)
M11[31:24] = (RN + 3)	M11[23:16] = (RN + 4)	M11[15:8] = (RN + 5)	M11[7:0] = (RN + 6)
M12[31:24] = (SS + 4)	M12[23:16] = (SS + 5)	M12[15:8] = (SS + 6)	M12[7:0] = (SS + 7)
M13[31:24] = FFh	M13[23:16] = FFh	M13[15:8] = FFh	M13[7:0] = 80h
M14[31:24] = 00h	M14[23:16] = 00h	M14[15:8] = 00h	M14[7:0] = 00h
M15[31:24] = 00h	M15[23:16] = 00h	M15[15:8] = 01h	M15[7:0] = B8h

注释:

Mt	SHA-1的输入; $0 \leq t \leq 15$; 32-位数
(SS + N)	密钥的第N个字节; 密钥从地址80h开始(见图5)
(PP + N)	数据页的第N个字节; 数据页从00h, 20h, 40h以及60h开始(见图5)
(SP + N)	暂存器的第N个字节
MP	MP[7:3] = 00000b, MP[2:0] = T[7:5]
(RN + N)	ROM注册码的第N个字节。ROM注册码的CRC字节不使用。

1Kb带SHA-1算法保护的1-Wire FRAM芯片

表 3B. Copy Scratchpad命令用于配置页时的SHA-1输入

M0[31:24] = (SS + 0)	M0[23:16] = (SS + 1)	M0[15:8] = (SS + 2)	M0[7:0] = (SS + 3)
M1[31:24] = (SS + 0)	M1[23:16] = (SS + 1)	M1[15:8] = (SS + 2)	M1[7:0] = (SS + 3)
M2[31:24] = (SS + 4)	M2[23:16] = (SS + 5)	M2[15:8] = (SS + 6)	M2[7:0] = (SS + 7)
M3[31:24] = (RP + 0)	M3[23:16] = (RP + 1)	M3[15:8] = (RP + 2)	M3[7:0] = (RP + 3)
M4[31:24] = (RP + 4)	M4[23:16] = (RP + 5)	M4[15:8] = (RP + 6)	M4[7:0] = (RP + 7)
M5[31:24] = (RP + 8)	M5[23:16] = (RP + 9)	M5[15:8] = (RP + 10)	M5[7:0] = (RP + 11)
M6[31:24] = (RP + 12)	M6[23:16] = (RP + 13)	M6[15:8] = (RP + 14)	M6[7:0] = (RP + 15)
M7[31:24] = (RP + 16)	M7[23:16] = (RP + 17)	M7[15:8] = (RP + 18)	M7[7:0] = (RP + 19)
M8[31:24] = (SP + 0)	M8[23:16] = (SP + 1)	M8[15:8] = (SP + 2)	M8[7:0] = (SP + 3)
M9[31:24] = (SP + 4)	M9[23:16] = (SP + 5)	M9[15:8] = (SP + 6)	M9[7:0] = (SP + 7)
M10[31:24] = MP	M10[23:16] = (RN + 0)	M10[15:8] = (RN + 1)	M10[7:0] = (RN + 2)
M11[31:24] = (RN + 3)	M11[23:16] = (RN + 4)	M11[15:8] = (RN + 5)	M11[7:0] = (RN + 6)
M12[31:24] = (SS + 4)	M12[23:16] = (SS + 5)	M12[15:8] = (SS + 6)	M12[7:0] = (SS + 7)
M13[31:24] = FFh	M13[23:16] = FFh	M13[15:8] = FFh	M13[7:0] = 80h
M14[31:24] = 00h	M14[23:16] = 00h	M14[15:8] = 00h	M14[7:0] = 00h
M15[31:24] = 00h	M15[23:16] = 00h	M15[15:8] = 01h	M15[7:0] = B8h

注释:

Mt	SHA-1的输入; $0 \leq t \leq 15$; 32-位数
(SS + N)	密钥的第N个字节; 密钥从地址80h开始(见图5)
(RP + N)	配置页的第N个字节; 配置页从88h开始(见图5) RP + 8到RP + 15 是ROM注册码
(SP + N)	暂存器的第N个字节
MP	MP[7:0] = 04h
(RN + N)	ROM注册码的第N个字节。ROM注册码的CRC字节不使用。

1Kb带SHA-1算法保护的1-Wire FRAM芯片

Read Authenticated Page [A5h]

Read Authenticated Page(读认证页数据)命令会使主机读取到部分数据和一个MAC认证值。主机可以通过这个MAC值来确定芯片的密钥,如果MAC值认证通过,那么芯片的密钥和主机持有的密钥一致。芯片计算MAC值时,输入包括密钥,指定页的所有数据,ROM注册码的前7个字节,以及一个5字节的询问数据,询问数据在暂存器中,启动Read Authenticated Page命令前,主机应该先用Write Scratchpad命令将询问数据写入暂存器。询问数据有5个字节,为暂存器的第3到第7个字节。或者主机也可以选择上一个命令后,留在暂存器中的数据作为询问数据。160位MAC从最低位开始传输,即E[0]开始,A[31]结束。Read Authenticated Page命令时的SHA-1输入见表4。

在主机发送命令后,继续发送指定地址(TA1和TA2),芯片接收到后将EN_LFS标志置0。如果指定的地址是有效的,即地址小于0080h,主机就会接收到指定页的数据,从开始的第一个到结束的最后一个,继续接收到1个FFh字节,以及一个反向的CRC字节,CRC字节由命令码,地址,数据和FFh字节生成。如果指定的地址无效,即地址大于等于0080h时,主机只能接收到FFh而非有效数据。接收到CRC后,主机应该开始等待一段tCSHA的算法计算时间,这段时间里单总线的电压值不能低于2.4V。算法计算时,芯片将密钥,指定页的所有数据,ROM注册码的前7个字节以及5字节的询问数据作为算法输入,最终计算出160位的MAC值。接着主机就可以读取这个MAC值,继续接收到一个反向的CRC值,这个CRC值可以验证160位MAC值传输是否正确。主机再继续接收数据,会接收到AAh。

表 4. Read Authenticated Page命令时的SHA-1输入

M0[31:24] = (SS + 0)	M0[23:16] = (SS + 1)	M0[15:8] = (SS + 2)	M0[7:0] = (SS + 3)
M1[31:24] = (PP + 0)	M1[23:16] = (PP + 1)	M1[15:8] = (PP + 2)	M1[7:0] = (PP + 3)
M2[31:24] = (PP + 4)	M2[23:16] = (PP + 5)	M2[15:8] = (PP + 6)	M2[7:0] = (PP + 7)
M3[31:24] = (PP + 8)	M3[23:16] = (PP + 9)	M3[15:8] = (PP + 10)	M3[7:0] = (PP + 11)
M4[31:24] = (PP + 12)	M4[23:16] = (PP + 13)	M4[15:8] = (PP + 14)	M4[7:0] = (PP + 15)
M5[31:24] = (PP + 16)	M5[23:16] = (PP + 17)	M5[15:8] = (PP + 18)	M5[7:0] = (PP + 19)
M6[31:24] = (PP + 20)	M6[23:16] = (PP + 21)	M6[15:8] = (PP + 22)	M6[7:0] = (PP + 23)
M7[31:24] = (PP + 24)	M7[23:16] = (PP + 25)	M7[15:8] = (PP + 26)	M7[7:0] = (PP + 27)
M8[31:24] = (PP + 28)	M8[23:16] = (PP + 29)	M8[15:8] = (PP + 30)	M8[7:0] = (PP + 31)
M9[31:24] = (SP + 2)	M9[23:16] = (SP + 3)	M9[15:8] = FFh	M9[7:0] = FFh
M10[31:24] = MP	M10[23:16] = (RN + 0)	M10[15:8] = (RN + 1)	M10[7:0] = (RN + 2)
M11[31:24] = (RN + 3)	M11[23:16] = (RN + 4)	M11[15:8] = (RN + 5)	M11[7:0] = (RN + 6)
M12[31:24] = (SS + 4)	M12[23:16] = (SS + 5)	M12[15:8] = (SS + 6)	M12[7:0] = (SS + 7)
M13[31:24] = (SP + 4)	M13[23:16] = (SP + 5)	M13[15:8] = (SP + 6)	M13[7:0] = 80h
M14[31:24] = 00h	M14[23:16] = 00h	M14[15:8] = 00h	M14[7:0] = 00h
M15[31:24] = 00h	M15[23:16] = 00h	M15[15:8] = 01h	M15[7:0] = B8h

注释:

Mt	SHA-1的输入; $0 \leq t \leq 15$; 32-位数
(SS + N)	密钥的第N个字节; 密钥从地址80h开始(见图5)
(PP + N)	数据页的第N个字节; 数据页从00h, 20h, 40h以及60h开始(见图5)
(SP + N)	暂存器的第N个字节
MP	MP[7:3] = 01000b, MP[2:0] = T[7:5]
(RN + N)	ROM注册码的第N个字节。ROM注册码的CRC字节不使用。

1Kb带SHA-1算法保护的1-Wire FRAM芯片

Anonymous Read Authenticated Page [CCh]

Anonymous Read Authenticated Page(匿名读认证页数据)命令是Read Authenticated Page(读认证页数据)命令的变形。两者的唯一区别在于SHA-1算法输入时的M10和M11(见表4)是FFh而不是ROM注册码。这样的话,生成的MAC值就不再和ROM ID挂钩,不再是某一个芯片的MAC认证值,就像是匿名了一样。

Refresh Scratchpad [A3h]

Refresh Scratchpad(刷新暂存器)命令将FRAM存储器数据读出到暂存器中,并且会将EN_LFS命令置1,之后可以通过Load First Secret命令将刚刚读出的数据重新写回到FRAM存储器中,不需要使用Copy Scratchpad命令,也不需要经过SHA-1算法的验证。Refresh Scratchpad命令的流程和Write Scratchpad命令的流程十分接近。对于0000h到007Fh的地址,主要有两个区别。第一,主机传输的8字节数据是无效数据,芯片不会将这些数据写入暂存器,而是将指定地址的FRAM数据原封不动地写入到暂存器中。如果指定地址的数据被读保护了,那芯片就会将主机刚发送的数据写入暂存器,防止读取到读保护的数据。第二,主机发送了八个无效数据字节后,EN_LFS标志会被置1。EN_LFS标志只有在以下情况才会被清0:在执行Write Scratchpad命令,(Anonymous) Read Authenticated Page命令, Refresh Scratchpad命令, Read Memory命令中接收到TA1和TA2字节后,或者上电复位后。这些情况能够清0EN_LFS标志的原因是,它们能够使得暂存器中的数据发生改变。如果数据被读保护了,EN_LFS标志也不会被置1。当指定的地址是0080h到009Fh时, Refresh Scratchpad命令的执行情况将和Write Scratchpad命令一模一样。这样可以防止之后的Read Scratchpad命令将密钥泄露。

Read Memory [F0h]

Read Memory(读存储器)命令可以将除了密钥以外的数据全部读出来。如果尝试读密钥数据,会读取到FFh而非真实数据。主机启动这个命令,接着指定地址,芯片首先将EN_LFS标志清0。如果指定的地址有效,主机就可以从指定的地址开始读取数据,一直读取数据直到9Fh地址。再继续往下读,就会一直读到逻辑1。值得注意的是,地址会随着数据的读取不停累加,地址寄存器的值指向最后读取的数据。E/S字节以及暂存器不受影响。

复位指令[CODEh]

复位指令为多个命令执行时的节点,用来组合不同的命令。芯片接收到复位指令后,会清空命令码寄存器,从而能够从上一条命令中跳出,执行下一条命令。例如在执行写暂存器后,需要发送一次复位指令,才能执行读暂存器命令。

复位指令也不会产生应答。

复位指令区别于芯片掉电后复位,掉电复位后整个系统全部复位,暂存器的数据清空,而复位指令只是复位部分电路,暂存器的数据不会清空。掉电复位后需要等待 t_{STD} 后,芯片系统稳定才能发送命令,而复位指令后可以直接发送命令。

复位指令用在写入类命令和读取类命令时有不同,写入类命令接收完CRC检验值后,需要多发8位的DUMMY bit再发送复位指令;读取类指令接收完CRC校验值后,不需要发送DUMMY bit,直接发送复位指令。

在有无限循环01应答,00应答的命令后,包括加载初始密钥,复制暂存器,生成下一密钥后,即可以改变FRAM内储存数据的命令后,不能再使用复位命令来组合下一个命令。这时要继续发送命令,必须执行一次掉电后复位操作。

例如一次写入FRAM密钥的过程,上电复位后先执行匹配ROM命令,发送命令码0Fh+8字节ROM注册码,接着执行写暂存器命令,发送命令码0Fh+2字节地址8000h,再发送8字节写入数据,数据发送后接收到CRC16校验值;继续发送8个DUMMY字节+复位指令码CODEh,发送Resume指令A5h+读暂存器指令AAh,读取到2字节地址+E/S字节,继续读取到8字节暂存器数据和CRC16校验,这时发送复位指令无需DUMMY字节;继续发送Resume命令+载入初始密钥命令5Ah,发送2字节地址+E/S字节,这时芯片开始写入80地址的密钥,等待tPROG后写入完毕,接收到AA无限循环的应答。成功写入FRAM后,就要重新掉电再上电以继续执行其他命令。

SHA-1 算法计算

此算法遵循美国国家标准与技术研究院的安全散列标准。算法的输入为16个32位数 $Mt(0 \leq t \leq 15)$, Compute Next Secret Command见表1, Copy Scratchpad见表3A及表3B, Read Authenticated Page见表4。算法含多个元素: 80个32位数 Wt 和 $Kt(0 \leq t \leq 79)$,逻辑函数 $f_t(B, C, D)$ ($0 \leq t \leq 79$),其中 B, C, D 为32位数,以及32位数 A, E, TMP 。逻辑运算有: 无进位加法运算("+"), 取反运算("v"), 异或运算("⊕"), 与运算("&"), 或运算("v"), 赋值运算(":="), 以及32位数的循环移位运算, 表达式" $Sn(X)$ "表示将32位数 X 向左循环移位 n 次。

1Kb带SHA-1算法保护的1-Wire FRAM芯片

函数 f_t 定义如下:

$$\begin{aligned} f_t(B,C,D) = & (B \wedge C) \vee ((B) \wedge D) & (0 \leq t \leq 19) \\ & B \oplus C \oplus D & (20 \leq t \leq 39) \\ & (B \wedge C) \vee (B \wedge D) \vee (C \wedge D) & (40 \leq t \leq 59) \\ & B \oplus C \oplus D & (60 \leq t \leq 79) \end{aligned}$$

数列 W_t ($0 \leq t \leq 79$) 定义如下:

$$\begin{aligned} W_t &:= M_t & (0 \leq t \leq 15) \\ &S^1(W_{t-3} \oplus W_{t-8} \oplus W_{t-14} \oplus W_{t-16}) & (16 \leq t \leq 79) \end{aligned}$$

数列 K_t ($0 \leq t \leq 79$) 定义如下:

$$\begin{aligned} K_t &:= 5A827999h & (0 \leq t \leq 19) \\ &6ED9EBA1h & (20 \leq t \leq 39) \\ &8F1BBCDCh & (40 \leq t \leq 59) \\ &CA62C1D6h & (60 \leq t \leq 79) \end{aligned}$$

变量 A, B, C, D, E 初始值为:

$$\begin{aligned} A &:= 67452301h \\ B &:= EFCDAB89h \\ C &:= 98BADCFEh \\ D &:= 10325476h \\ E &:= C3D2E1F0h \end{aligned}$$

算法结果160位MAC为A, B, C, D, E经过80轮($t = 0-79$)循环计算后得到的值 (舍弃所有进位):

$$\begin{aligned} TMP &:= S^5(A) + f_t(B,C,D) + W_t + K_t + E \\ E &:= D \\ D &:= C \\ C &:= S^{30}(B) \\ B &:= A \\ A &:= TMP \end{aligned}$$

通过(Anonymous) Read Authenticated Page 命令, 主机可以读取到一个算法结果的MAC值, 传输的顺序见表2. 输出结果从E[0]开始传输, 到A[31]结束。使用 Compute Next Secret命令不会暴露MAC值, D和E两个变量在计算完成后会直接复制到密钥区, 见表1.

注意芯片的SHA-1计算流程和标准算法的流程有差别, 标准的计算流程中, 最后计算的变量ABCDE值, 要加上初始值, 而芯片的算法流程省略了这一步加法。因为芯片的算法输入都是512位, 每次计算都只有80轮循环, 最后一步的加法不再加强保密功能, 所以将这一步加法删除了。如果使用某些软件函数, 遵循的是标准的计算流程, 算出来的结果应该减去变量ACBCDE的初始值。

1-Wire Bus System

Hardware Configuration

1Kb带SHA-1算法保护的1-Wire FRAM 芯片

流程顺序

PRB480的流程按照如下顺序进行:

- 初始化
- ROM功能命令
- 存储器/SHA-1功能命令
- 命令执行、数据传输

初始化

所有命令执行前都必须进行初始化，主机通过复位来进行初始化。复位分为两种，一种是断电后的上电复位，一种是主机发送的复位指令。为了适应没有电容的使用环境，复位不会产生应答，复位后等待时间Tstd后，芯片系统稳定，芯片做好了接收和执行各种命令的准备，这时主机可以开始发送命令。

ROM功能命令

复位后等待芯片系统稳定，主机首先可以开始发送ROM功能命令。所有的ROM功能命令都是8位的，具体描述见下文。图10为ROM功能命令的流程图。

Read ROM [33h]

Read ROM (读取ROM注册码)命令可以获取芯片的64位ROM 注册码，其中包括8位家族码，48位序列号以及8位CRC校验值。获取ROM注册码后，RC标志不会置1，RC标志为Resume Command(继续命令)标志，该标志指示Resume 命令是否可用，后续Resume命令介绍时会有更多详情。

Match ROM [55h]

Match ROM(匹配ROM)命令执行时，命令后接着发送64位ROM注册码，主机发送的64位ROM注册码和芯片的64位注册码完全匹配后，RC标志会被置1。

Skip ROM [CCh]

Skip ROM(跳过ROM)命令可以跳过ROM注册码的读取和配对，从而节省时间，接下来可以直接执行存储器功能命令或者SHA-1算法相关命令。RC标志不会置1。

1Kb带SHA-1算法保护的1-Wire FRAM芯片

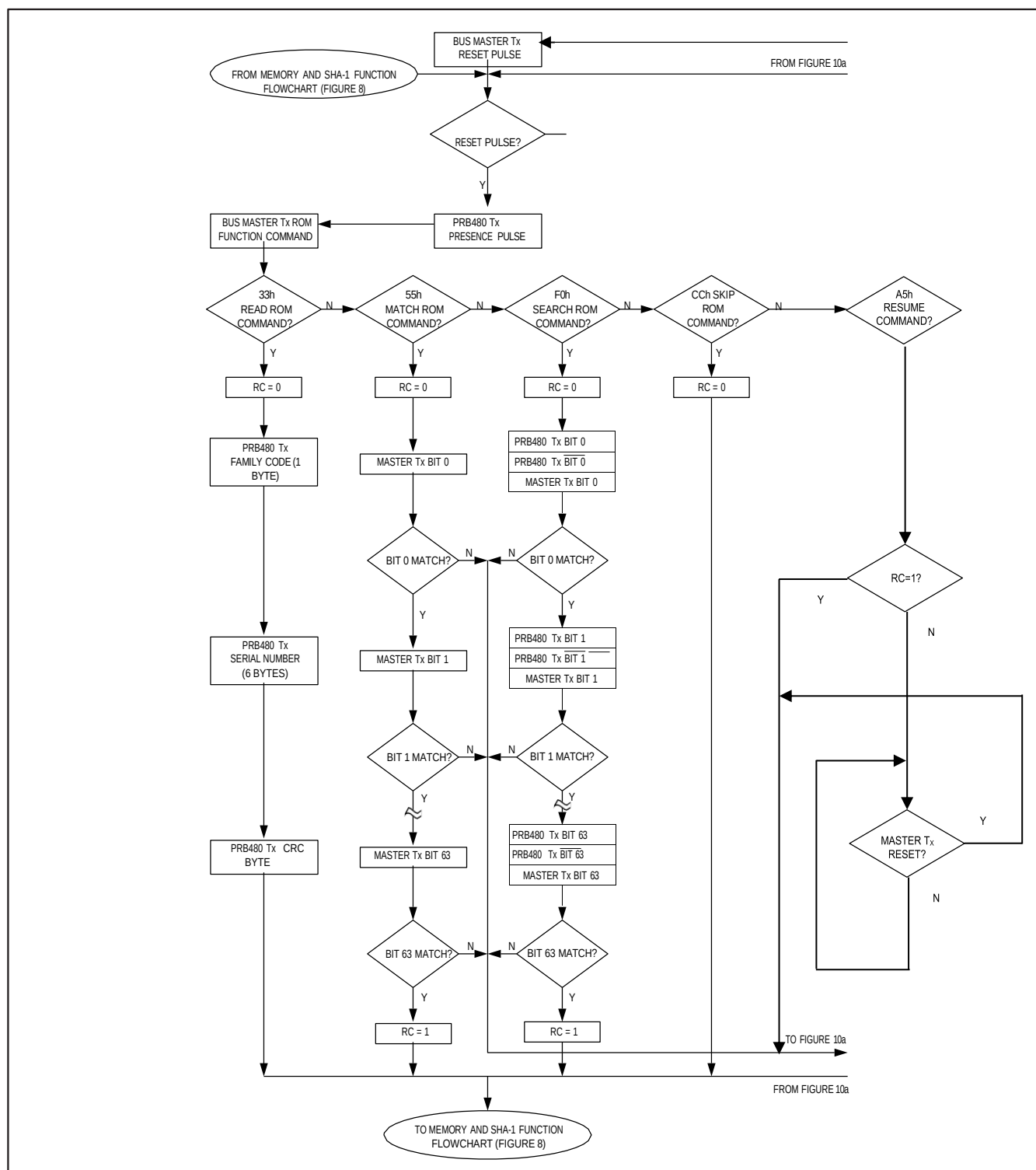


图 10a. ROM 功能流程图

1Kb带SHA-1算法保护的1-Wire FRAM芯片

Resume [A5h]

为了节省传输时间，芯片设计了Resume(继续)命令。这个命令执行时会检查RC标志，只有在RC标志为1时才能生效，RC标志只有在成功执行Match ROM命令后才会被置1，即主机和芯片已经完成了一次成功的ROM 注册码匹配。RC标志置1后，只有芯片没有掉电，就可以重复地使用Resume 命令来访问存储器和SHA-1算法。复位指令不会清除RC标志，但是掉电后重新上电，RC标志会清0。

1-Wire 通讯

主机和芯片之间的通信只需要一根数据总线，和一根返回地线。为了保证通信的准确性，需要明确以下的通信行为：复位脉冲，写1时隙，写0时隙以及读数据时隙。值得注意的是，所有的通信行为都是由主机拉低总线开始的。PRB480的通信速度为125Kbit/s。

从闲置状态到激活状态，单总线上的电压需要从 V_{PUP} 降低到 V_{TL} 。从激活状态到闲置状态，单总线上的电压需要从 V_{ILMAX} 上升到 V_{TH} 。

电压 V_{ILMAX} 和低电平的定义有关。图11展示的是芯片掉电后上电复位的过程，主机将芯片拉低 t_{RSTL} 后，确保芯片完全掉电，接着又重新上电。上电后，需要等待 t_{STD} 的时间，芯片系统稳定后，主机才能开始发送各种命令。与标准的单总线协议有区别的是，为了适应电容很小的工作环境，芯片不会产生复位应答。

1Kb带SHA-1算法保护的1-Wire FRAM芯片

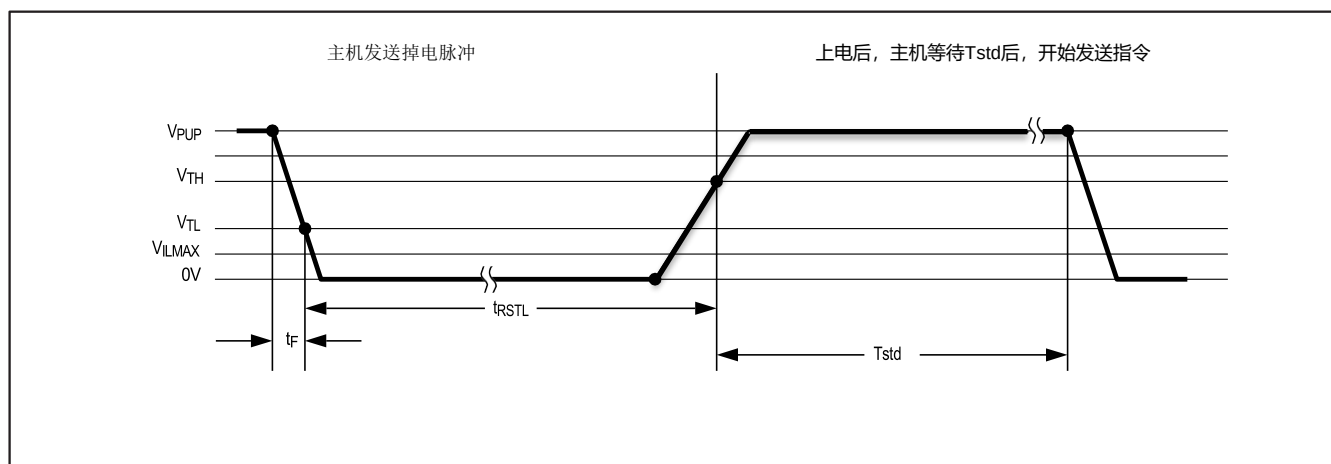


图 11. 初始化过程:掉电后上电复位

读写时序

从机到主机

PRB480芯片和主机之间的数据通讯以时隙为基本单位，每个时隙代表1比特数据。写时隙由主机向芯片发送，读时隙由芯片向主机发送。图12展示了写1时隙，写0时隙以及读数据0时隙。

所有的通讯过程都由主机拉低总线开始，在总线电压低于 V_{TH} 后，芯片内部时钟开始计时，并通过采样来判断当前读取的数据是逻辑1或者逻辑0，以及决定数据读出时的有效时间长短。

读取数据的时隙也是以主机下拉开始。主机下拉总线到 V_{TL} 后，等待 t_1 时间以准备开始采样，采样时间持续 t_{SAMPLE} ，如果是数据0，芯片会在采样时间内拉低一个窄脉冲到 V_0 ，持续时间为 t_{READ0} ；如果是数据1，则不会产生这样的窄脉冲。

主机在 t_{READ0} 期间进行采样，采样到总线拉低的动作，就解析出逻辑0，反之没有就解析出逻辑1。采样时间过去后，主机应该在 t_2 时间内上拉总线。

主机到从机

发送一个写1的时隙，主机要发送1个向下的脉冲，首先必须将总线电压拉低到至少 V_{th} ，且低电平持续时间必须超过 t_{LOW} 。

发送一个写0的时隙，主机要发送2个向下的脉冲，两个脉冲的低电压都必须达到 V_{th} ，低电平持续时间超过 t_{LOW} ，两个低电平脉冲之间的间隔不能超过 t_{GAP} 。

1Kb带SHA-1算法保护的1-Wire FRAM芯片

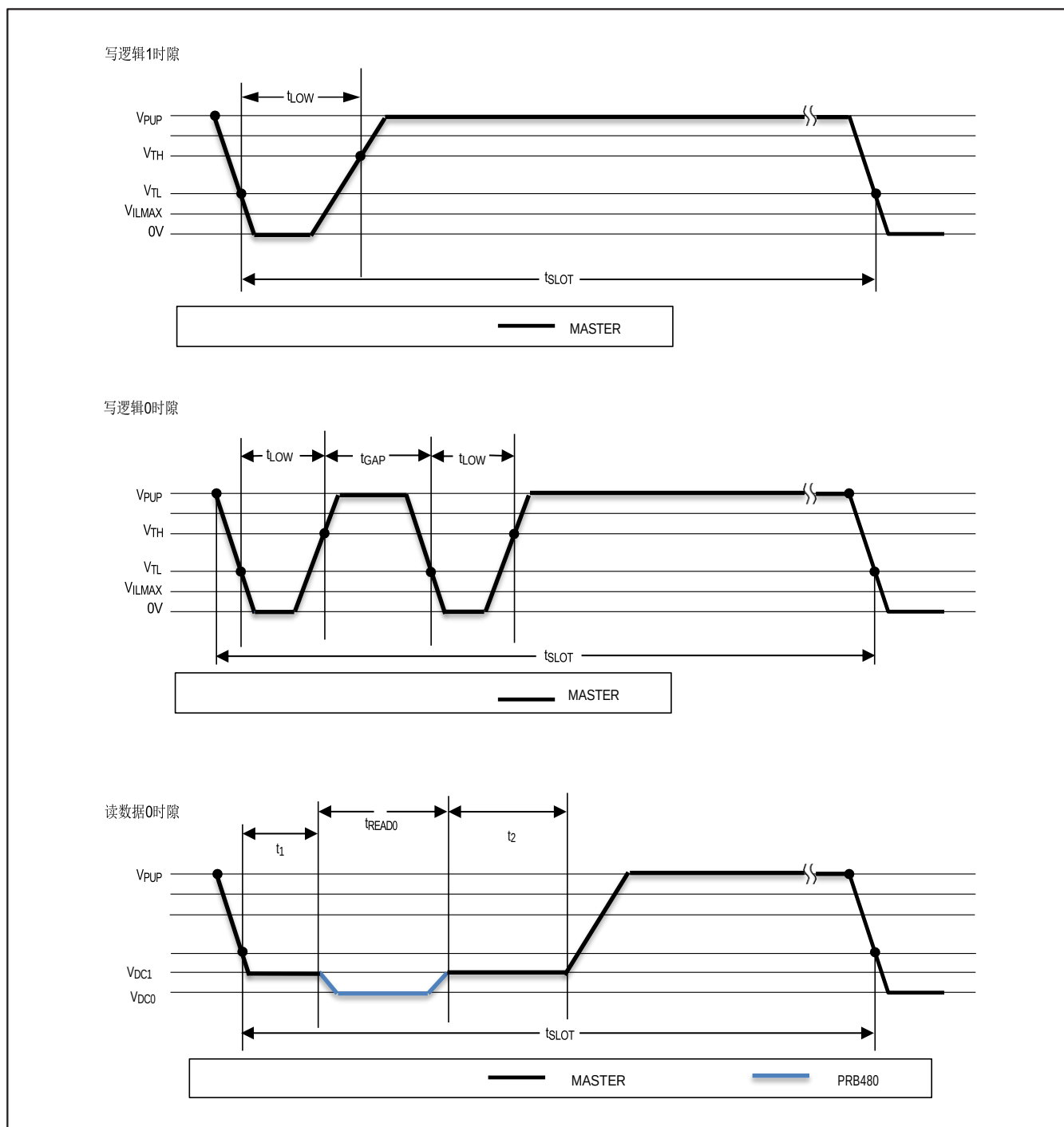


图12. 读写时序图

1Kb带SHA-1算法保护的1-Wire FRAM芯片

CRC 校验

PRB480有两种CRC校验，一种是8位的CRC，由ROM注册码的前七个字节计算产生，主机接收到ROM注册码后，根据CRC计算多项式 $X^8+X^5+X^4+1$ ，计算出8位校验值，以此来判断接收到的ROM注册码数据是否完整有效。这8位的CRC校验值就是计算得到的值，不需要取反。

另外一种CRC校验值是16位的，计算多项式是标准的 $X^{16}+X^{15}+X^2+1$ 。这种CRC用来校验读写暂存器，读写存储器或者读取MAC值时的数据传输是否准确无误。和8位的CRC不同，16位的CRC是以反相的结果进行传输的。图14展示了16位CRC的计算过程，不同命令的CRC计算输入见各个命令的流程图(图8)。主机计算出

CRC值和接收到芯片发送的CRC值进行比较，如果CRC校验结果一致，那么可以继续进行后续操作；如果不一致，那么数据传输过程有误，应该重新进行上一次的数据传输操作，直到CRC校验通过。

对于Write Scratchpad命令和Refresh Scratchpad命令，芯片首先清零CRC计算器，接着依次移入命令码，地址TA1和TA2(T[2:0]置为0)，主机发送的所有数据。芯片在接收到整8个字节数据后才送出CRC值。

对于Read Scratchpad命令，芯片首先清零CRC计算器，接着依次移入命令码，地址TA1和TA2，E/S字节，暂存器数据(暂存器数据可能在上一次的Write Scratchpad命令中被写入)。芯片在读取完整个暂存器数据后才送出CRC值。

1Kb带SHA-1算法保护的1-Wire FRAM芯片

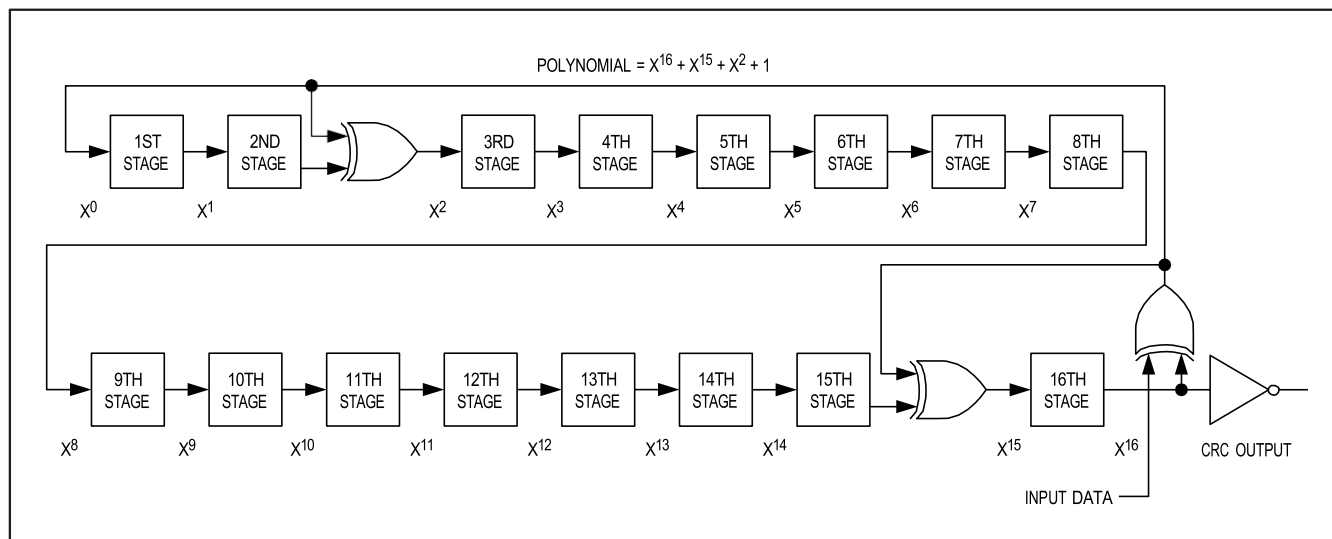


图14. CRC-16 计算过程及计算多项式

对于(Anonymous) Read Authenticated Page命令，芯片首先清零CRC计算器，接着依次移入命令码，两字节的地址，所有的数据字节，以及一个FF字节。获取MAC后的CRC计算也是从清零CRC计算器开始，接着移入160位的MAC值，MAC移入的顺序和主机接收到的顺序一致。

举例说明，假如芯片的ROM注册码前七个字节数据是0xAF, 0xF0, 0x10, 0x04, 0xA0, 0x31, 0x03，那么8位CRC校验值为0xC7。

从总线上获取的数据依次为11100011。

在一次写暂存器的操作中，命令码是0x0F，地址是0x1000，写入的8字节数据是：

0x18,0x29,0x3f,0x4e,0x5d,0x6c,0x7b,0x8a，

计算器生成的16位CRC校验值为0xF16B，从总线上接收到的数据依次为1101011010001111。

通讯行为表格

简称	描述
RST	由主机产生的复位
Select	满足ROM功能的数据和命令.
WSP	Write Scratchpad 写暂存器命令
RSP	Read Scratchpad 读暂存器命令
LFS	Load First Secret 加载初始密钥命令
CNS	Compute Next Secret 生成下一密钥命令
CPS	Copy Scratchpad 复制暂存器命令
RAP	Read Authenticated Page 读认证页数据命令
ARAP	Anonymous Read Authenticated Page 匿名读认证页数据
RFSP	Refresh Scratchpad 刷新暂存器命令

1Kb带SHA-1算法保护的1-Wire FRAM芯片

通讯行为表格(继续上面的表格)

简称	描述
RM	Read Memory 读存储器命令
TA	地址 TA1, TA2.
TA-E/S	地址 TA1, TA2 和 E/S 字节
<8 bytes>	传输8字节数据
<data to EOM>	End of the memory 传输尽可能多的数据直到存储器末尾
CRC-16	传输反向的16位CRC校验值
<data>, FF, CRC-16	传输若干数据, 1个FF字节, 以及16位反向的CRC校验值
MAC	Message Authentication Code 传输MAC认证码, 160位数据SHA-1计算结果
00 loop	主机读取到无限的00循环
FF loop	主机读取到无限的FF循环
AA loop	主机读取到无限的AA循环
Compute SHA-1	芯片内部启动SHA-1引擎以计算MAC认证码, 这段时间里主机不能发送别的操作
Programming	数据正在写入FRAM中, 这段时间里主机不能发送别的操作

不同颜色代表的通讯行为

主机到从机	从机到主机	Compute SHA-1	Programming
-------	-------	---------------	-------------

一系列通讯行为举例

写暂存器(成功)

RST	Select	WSP	TA	<8 Bytes>	CRC-16	FF Loop
-----	--------	-----	----	-----------	--------	---------

写暂存器(失败, 无效地址)

RST	Select	WSP	TA	FF Loop
-----	--------	-----	----	---------

读暂存器(必然成功)

RST	Select	RSP	TA-E/S	<8 Bytes>	CRC-16	FF Loop
-----	--------	-----	--------	-----------	--------	---------

载入初始密钥(成功)

RST	Select	LFS	TA-E/S	Wait t _{PROG} (MAX)	AA Loop
-----	--------	-----	--------	------------------------------	---------

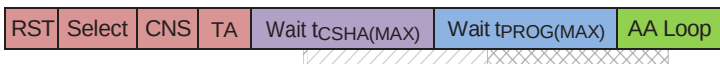
载入初始密钥(失败, 无效的地址或写保护)

RST	Select	LFS	TA-E/S	FF Loop
-----	--------	-----	--------	---------

1Kb带SHA-1算法保护的1-Wire FRAM芯片

一系列通讯行为举例

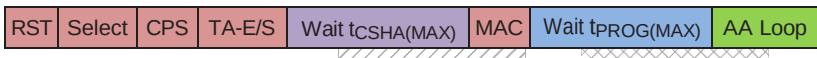
生成下一密钥 (成功)



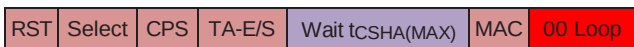
生成下一密钥 (失败, 无效的地址或写保护)



复制暂存器 (成功)



复制暂存器 (失败, MAC不匹配)



复制暂存器 (失败, 无效的地址或写保护)



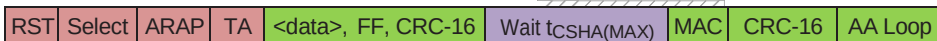
读认证页数据 (成功)



读认证页数据 (失败, 无效的地址)



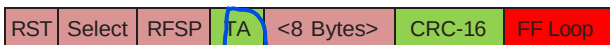
匿名读认证页数据 (成功)



匿名读认证页数据 (失败, 无效的地址)



刷新暂存器 (成功)



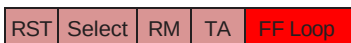
刷新暂存器 (失败, 无效的地址)



读存储器 (成功)



读存储器 (失败, 无效的地址)



猪肝色 主机给从机

1Kb带SHA-1算法保护的1-Wire FRAM 芯片

管脚示意图

待补充

封装示意图

待补充

封装信息

待补充

1Kb带SHA-1算法保护的1-Wire FRAM 芯片

版本信息

日期	描述	更新
2025/09/28	版本V0.0	